

TDA Bootcamp – Deep Learning

Face Recognition Project

Saksham Seth – 240968166

- Face detection was implemented using the YOLOv8n model from the Ultralytics library, configured to detect only class 0 (persons). The model's bounding box output was processed using numpy to identify the largest detected face based on area, and the corresponding region was extracted directly from the original frame using array slicing.
- Face registration was handled by capturing a single frame from the system webcam via OpenCV. The detected face region was cropped and saved to the local filesystem under the `registered_faces/` directory using the provided username as the filename. The face image was stored in .jpg format for compatibility.
- Face recognition leveraged the DeepFace library using the `verify` function with the Facenet embedding model. Input consisted of the live face image captured via webcam and each registered face image. Face embeddings were compared via L2 norm, and verification passed if the computed distance was below a strict threshold (0.3). Detection enforcement was disabled to avoid breaking on imperfect crops.
- The UI was constructed using Streamlit for rapid prototyping and lightweight deployment. It included a sidebar-based navigation menu for user registration and authentication, a live webcam preview during capture, and dynamic status updates for success/failure. The UI also utilized Streamlit's session state and image previewing to enhance user feedback during live capture.

- False acceptances occurred when the subject and an unregistered user shared environmental similarities (e.g., identical backgrounds), highlighting the reliance on contextual cues in the absence of robust embedding discrimination. Conversely, false rejections were observed under changes in illumination, angle, or appearance (e.g., hairstyle), reflecting the sensitivity of Facenet embeddings to intra-class variance.
- Potential improvements include storing multiple embedding samples per user, using an ensemble model or embedding averaging to reduce variance, applying histogram equalization or CLAHE pre-processing to normalize lighting, and incorporating a liveness detection model or anti-spoofing mechanism to reduce vulnerability to presentation attacks.